

FTCTeamHub — Repo Setup, Xcode Project, and Windows→AltStore Deployment

Important reality check first

Building an iOS app requires Apple's Xcode toolchain, which only runs on macOS. You're on Windows, so you have three real options — pick one before continuing:

1. Borrow/rent a Mac (a friend's Mac, a school Mac lab, or a cloud Mac like MacinCloud/MacStadium) to open Xcode, build, and export the .ipa.
2. Use a GitHub Actions macOS runner to build the .ipa in the cloud, using free-tier automatic signing tied to your Apple ID. This is fully scriptable from Windows and is what the workflow below sets up.
3. Ask a teammate with a Mac to do the local build once, then you take over CI/GitHub duties.

Once you have a signed .ipa (from either path), sideloading it onto your iPhone via AltStore on Windows is straightforward and is covered in Part 3.

Part 1 — Create the GitHub repo (from Windows)

1. Go to github.com/new, name the repo FTCTeamHub, set it **Private** (recommended — it'll contain your Apple ID's provisioning artifacts), and check "Add a README."
2. On your Windows machine, install [Git for Windows](#) and [GitHub Desktop](#) (easiest GUI option) — or just use the git CLI if you prefer.
3. Clone the repo locally:

```
git clone https://github.com/<your-username>/FTCTeamHub.git
cd FTCTeamHub
```

4. Create this folder structure (matches standard Xcode project layout so it drops in cleanly once you're on a Mac):

```
FTCTeamHub/
  FTCTeamHub.xcodeproj/      (generated by Xcode later — do NOT hand-create)
  FTCTeamHub/
    App/
      FTCTeamHubApp.swift
    Models/
      Models.swift
    Services/
      FTCScoutAPIClient.swift
      BackendServices.swift
    Views/
      ScoutingTabView.swift
      TasksTabView.swift
```

```
NotebookTabView.swift
IdeasTabView.swift
ActivityTabView.swift
Assets.xcassets/
Info.plist
.github/
  workflows/
    build-ipa.yml
.gitignore
README.md
```

5. Drop each `.swift` file I generated into its matching folder above (all seven files are attached to this conversation — copy their contents in).
6. Add a standard Swift `.gitignore` (search "github swift gitignore" or use GitHub's template) so you don't commit `xcuserdata/`, `DerivedData/`, or `.ipa` build artifacts.
7. Commit and push:

```
git add .
git commit -m "Initial FTCTeamHub architecture: models, services, five tab modules"
git push
```

Part 2 — Turning this into a real Xcode project (needs a Mac, even briefly)

You cannot generate a valid `.xcodproj` from Windows — Xcode's project file format isn't hand-editable reliably. When you get access to a Mac (even for 20 minutes at a library/school, or via MacinCloud's hourly plans):

1. Open Xcode → **File** → **New** → **Project** → **iOS** → **App**. Name it `FTCTeamHub`, interface: `SwiftUI`, language: `Swift`, uncheck `Core Data` (we use `SwiftData`), minimum deployment target `iOS 17.0` (required for `@Observable/SwiftData`).
2. Delete the auto-generated `ContentView.swift` and default `App.swift`.
3. Drag the folders from your cloned `FTCTeamHub` repo (`Models`, `Services`, `Views`, `App`) into the Xcode project navigator, checking "Copy items if needed" and "Create groups."
4. In the target's **Signing & Capabilities** tab: check "Automatically manage signing," select your personal Apple ID team (free accounts work for sideloading — apps just expire after 7 days and need re-signing).
5. Build once (`Cmd+B`) to confirm it compiles, then commit the generated `.xcodproj` and push it back to GitHub from the Mac (or copy it back to your Windows clone).

Part 3 — Building the IPA via GitHub Actions (so Windows can drive it going forward)

Once the `.xcodproj` exists in the repo, add this workflow so future builds happen in the cloud without needing a Mac each time. Note: free Apple ID signing is device-and-time limited and GitHub-hosted signing with a personal team requires exporting your certificate/provisioning profile as repo secrets — this is genuinely one of the fiddlier parts of iOS CI. A simpler alternative many FTC teams use: keep builds manual on a borrowed Mac, and only use GitHub for source control + code review, not CI.

If you do want CI, here's the workflow skeleton ([.github/workflows/build-ipa.yml](https://github.com/actions/workflows/build-ipa.yml)):

```
name: Build IPA
on:
  workflow_dispatch:
jobs:
  build:
    runs-on: macos-14
    steps:
      - uses: actions/checkout@v4
      - name: Select Xcode
        run: sudo xcode-select -s /Applications/Xcode_15.4.app
      - name: Build archive
        run: |
          xcodebuild archive \
            -project FTCTeamHub.xcodeproj \
            -scheme FTCTeamHub \
            -archivePath build/FTCTeamHub.xcarchive \
            -allowProvisioningUpdates \
            CODE_SIGN_STYLE=Automatic
      - name: Export IPA
        run: |
          xcodebuild -exportArchive \
            -archivePath build/FTCTeamHub.xcarchive \
            -exportPath build/ipa \
            -exportOptionsPlist ExportOptions.plist
      - uses: actions/upload-artifact@v4
        with:
          name: FTCTeamHub-ipa
          path: build/ipa/*.ipa
```

You'll need an `ExportOptions.plist` (method: development) and signing secrets set up via `fastlane match` or manual certificate export — this part genuinely benefits from a mentor or a Mac-owning teammate walking through it once.

Part 4 — Sideload the IPA with AltStore (Windows)

Once you have the `.ipa` (downloaded from the GitHub Actions artifact, or exported from Xcode on a Mac):

1. Install **iTunes/iCloud for Windows** first (AltServer depends on Apple's device drivers) [web:10].
2. Download **AltServer** for Windows from altstore.io and install it — it runs quietly in your system tray.
3. Plug your iPhone into your Windows PC via USB (or connect over Wi-Fi once paired).
4. In the AltServer tray icon menu, hold **Shift** and click the icon to reveal "Install AltStore", then select your device — this installs the AltStore app itself onto your phone [web:3].
5. On your iPhone: **Settings** → **General** → **VPN & Device Management** → trust your Apple ID developer profile.

6. Open AltStore on your iPhone, go to the My Apps tab, tap the + button, and browse to your FTCTeamHub.ipa file (transfer it to the phone first via AirDrop-alternative, iCloud Drive, or a USB file transfer app, since AltServer on Windows lets you pick a local .ipa path directly when installing) [web:15][web:3].
7. AltStore signs and installs the app using your Apple ID. Free accounts require AltServer to re-sign the app every 7 days — keep AltServer running on your PC and your phone on the same Wi-Fi network periodically, and AltStore will auto-refresh it in the background.

Part 5 — Team collaboration workflow going forward

- Every teammate contributing code should clone the repo, create a feature branch (`git checkout -b feature/kanban-drag-drop`), and open a Pull Request rather than pushing straight to `main` — this keeps your MVVM boundaries clean as more people touch Views vs. Services.
- Wire up real Firebase Auth/Firestore or Supabase (swap `MockBackendService` for a `FirebaseBackendService` conforming to the same `AuthServicing/SyncServicing` protocols) once you're ready for real multi-device sync — the protocol boundary in `BackendServices.swift` means no View code changes.
- Keep `Assets.xcassets`, provisioning profiles, and any .p12 certificates out of git history in a public repo; use repo secrets for CI.

If you get stuck at the Xcode-project-generation step specifically because you have zero Mac access, tell me — the practical fallback most FTC software leads use is a \$1/hr MacinCloud session just to do Part 2 once, after which GitHub + Windows handles everything else.